# SAE 1.03 – Installation d'un poste pour le développement

Dartevelle Samuel - Rigal Geoffrey

Compétences apprises / approfondies :

- Initialisation d'une machine virtuelle (Debian)
- Développement en Bash
- Fonctionnement système d'exploitation sous Linux

# Sujet

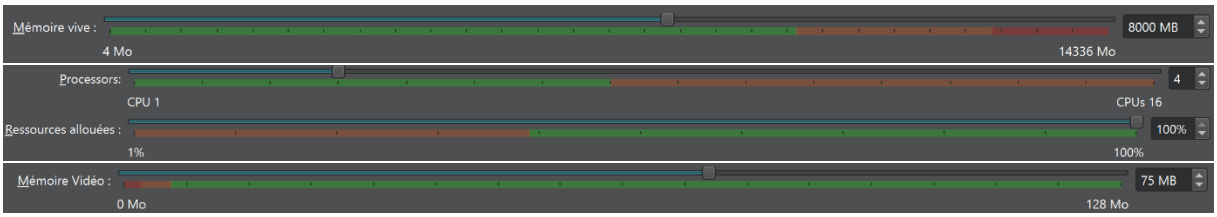
Dans l'entreprise de formation « ButIné », nous étions chargés de l'installation d'un environnement de développement sous linux (Debian) sur les postes informatiques.

Nous devions gérer les droits et les espaces selon l'année de formation, et donc créer des groupes avec des utilisateurs et un espace dédié.

Pour finir, nous étions en charge de créer un système de gestion de bibliothèque pour gérer les emprunts, les restitutions, les inscriptions d'adhérents, les nouveaux livres, etc...

# 1ere étape : Installation de Debian

Nous avons téléchargé l'image de disque (ISO) de la dernière distribution Debian puis avons alloué les ressources nécessaires au bon fonctionnement de la machine.




Lors de l'initialisation, nous avions alloué 16go de RAM. Seulement, nous avons importés la machine virtuelle sur nos ordinateurs personnels, il n'était donc pas possible d'en allouer autant sur nos PC personnels, d'où les 8go de RAM.

# 2ème étape : Création des comptes

Il a fallu avant tout obtenir les droits de superuser sur notre compte « Samrig » pour manipuler les groupes.

Grace à la commande addgroup, nous avons pu créer les groupes annee1, 2 et 3.

Grace à la commande useradd, nous avons pu créer les utilisateurs test1, 2 et 3 dans leur groupe respectif

Ensuite on crée les dossiers dans lesquels les élèves des groupes pourront manipuler les fichiers.

Pour finir, on attribue les droits de lecture, écriture et exécution pour l'utilisateur, le groupe mais pas les autres.

```
####### Gestion des groupes
sudo addgroup annee1
sudo addgroup annee2
sudo addgroup annee3
sudo useradd -m -g annee1 test1
sudo useradd -m -g annee2 test2
sudo useradd -m -g annee3 test3
mkdir /home/formation_annee1 /home/formation_annee2 /home/formation_annee3
####### Gestion des droits
sudo chown :annee1 /home/formation_annee1
sudo chmod 770 /home/formation_annee1
sudo chown :annee2 /home/formation_annee2
sudo chmod 770 /home/formation_annee2
sudo chown :annee3 /home/formation_annee3
sudo chmod 770 /home/formation_annee3
```

## 3ème étape : Gestion de la bibliothèque

Sur l'ensemble des programmes à réaliser, les difficultés majeures que nous avons rencontrées sont les suivantes :

- La compréhension des mécaniques et utilité des fichiers
- Gestion et calculs des dates
- Lire et modifier des fichiers

Pour le fichier *inscrire.bash*, nous avons dans un premier temps organisé le programme autour de l'inscription d'un adhérent ou d'un ouvrage (-a ou -l en argument). En fonction, on adapte l'utilisation de la commande, notamment le nombre d'argument utilisé.

Pour le fichier *demande.bash*, nous effectuons dans un premier temps les vérifications nécessaires au bon fonctionnement du programme. Il faut vérifier que les arguments placés dans la commande soient bien dans la base de données. On en profite pour récupérer des informations qui nous servirons plus tard.
Si toutes les vérifications sont passées, on effectue la demande d'emprunt en modifiant le fichier « emprunts » et « exemplaires ».

Il suffit dans l'idée de réutiliser le fichier *demande.bash* et d'en inverser le fonctionnement pour créer *rend.bash*. Il suffit de vérifier si le livre et l'adhérent existent dans la base de données. Si oui, on modifie les fichiers nécessaires pour effectuer l'action.

Pour le fichier *comptemps.bash*, nous vérifions que les arguments fournis soient corrects. Puis on récupère dans des tableaux la date au moment de l'exécution du programme et la date fournie en argument sous cette forme : [ jour, mois, année, heure:minute:secondes ]. Dans notre programme, nous avons choisi de traiter le nombre de jours de chaque mois. Nous calculons ensuite la différence entre la date réelle et la date fournie en commençant par les jours en gardant la retenue sur le mois si besoin. Nous répétons cette opération sur les mois en gardant une retenue sur l'année si besoin, et enfin on calcule la différence des années.
Enfin on compare les résultats obtenus pour déterminer la valeur à écrire dans la console.

Après avoir vérifié si aucun argument n'est placé, *rappel.bash* parcourt tous les emprunts en cours dans le fichier « emprunts ». En s'appuyant sur *comptemps.bash* on renvoie le nom de tous les emprunts datant de plus d'un mois en affichant le nom de l'adhérent concerné et l'ouvrage emprunté.

## Conclusion :

Sur ce projet, nous avons approfondis nos connaissances en développement bash, en manipulation de fichiers avec des commandes linux, en gestion des droits d'accès et appris à mettre en route une machine virtuelle sous Debian en prenant en compte les spécificités techniques d'un ordinateur.

Cependant notre projet n'est pas parfait. Il reste des gestions d'erreurs pas traitées ce qui peut causer des plantages. Également, nous avons rencontrés des difficultés quant à la lenteur des programmes qui subsistent toujours. Ce n'est pas dérangeant quand on a 5 adhérents et 10 livres mais dans le cadre d'une grande bibliothèque, c'est handicapant.

Si nous avions plus de temps, nous ferions une plus grande couverture sur les gestions d'erreur et chercherions à optimiser nos programmes.

# Programmes :

**demande.bash :**

```
#!/bin/bash

# Verification du nombre d'argument
if [[ $# -ne 2 ]]; then
    echo -e "Erreur 1 : Le nombre d'arguments n'est pas correct. \n Usage : $0
nom titre"
    exit 1
fi

# Verification du nom dans la BDD
verif_nom=0
for lignes in $(cat ../Bdd/membres) # Parcours du fichier membres
do
    nom=$(echo "$lignes" | cut -d ";" -f 2) # Recupération du nom de famille
    if [[ "$nom" == "$1" ]]; then # Si le nom est trouvé dans la BDD
        verif_nom=1
        num_adherant=$(echo "$lignes" | cut -d ";" -f 1)
    fi
done

# Verification du titre dans la BDD
verif_titre=0
for lignes in $(cat ../Bdd/livres)
do
    titre=$(echo "$lignes" | cut -d ";" -f 2)
    if [[ "$titre" == "$2" ]]; then
        verif_titre=1
        num_livre=$(echo "$lignes" | cut -d ";" -f 1)
    fi
done

# Verification de l'exemplaire du titre dans la BDD
verif_dispo=0
for lignes in $(cat ../Bdd/exemplaires)
do
    num_exemplaire=$(echo "$lignes" | cut -d ";" -f 1)
    if [[ "$num_exemplaire" == "$num_livre" ]]; then
        disponibilite=$(echo "$lignes" | cut -d ";" -f 3)
        if [[ "$disponibilite" == "oui" ]]; then
            verif_dispo=1
            break
        fi
    fi
```

```
done

# Test vérification des conditions
if [[ $verif_nom -eq 0 ]]; then
    echo "Le nom de l'adhérent est eronné ou n'existe pas dans la base de
donnée."
elif [[ $verif_titre -eq 0 ]]; then
    echo "Le nom du titre est eronné ou n'existe pas dans la base de donnée."
elif [[ $verif_dispo -eq 0 ]]; then
    echo "Aucun exemplaire de $2 n'est disponible."
else # Ecriture des differents fichiers
    date=$(date +"%d %m %Y %H:%M:%S")
    num_exemplaire_a_ecrire=$(echo "$lignes" | cut -d ";" -f 2)
    echo "$num_adherant;$num_livre;$num_exemplaire_a_ecrire;$date" >>
../Bdd/emprunts
    sed -i
"s/$num_livre;$num_exemplaire_a_ecrire;oui/$num_livre;$num_exemplaire_a_ecrire
;non/g" ../Bdd/exemplaires # Mise a jour du fichier exemplaires

    echo "l'exemplaire $num_exemplaire_a_ecrire du livre $2 à bien été
emprunté par $1"
fi
```

**comptemps.bash**

```
#!/bin/bash

# Verification du nombre d'arguments
if [[ $# -ne 4 ]]; then
    echo -e "Erreur 1 : Le nombre d'arguments n'est pas correct. \n Usage : $0
jour mois année heure:minute:secondes"
    exit 1
fi

# Vérification des valeurs des arguments
if [[ $1 -lt 1 || $1 -gt 31 ]]; then
    echo -e "Erreur 2 : La valeur du jour n'est pas correcte. \n Doit être
comprise entre 1 et 31"
    exit 2
elif [[ $2 -lt 1 || $2 -gt 12 ]]; then
    echo -e "Erreur 3 : La valeur du mois n'est pas correcte. \n Doit être
comprise entre 1 et 12"
    exit 3
else
    # Gestion des dates
    date_brute=$(date +"%-d %-m %Y")
    date_actu=($date_brute)
    date_args=($1 $2 $3)
```

```
    # Nombre de jours dans une année
    mois=(31 28 31 30 31 30 31 31 30 31 30 31)

    retenue_mois=0
    retenue_annee=0

    # Calcul jours
    diff_jour=$(( date_actu[0] - date_args[0] ))

    if [[ $diff_jour -lt 0 ]]; then
        index_mois_prec=$(( date_actu[1] - 2 ))
        [[ $index_mois_prec -lt 0 ]] && index_mois_prec=11

        diff_jour=$(( diff_jour + ${mois[$index_mois_prec]} ))
        retenue_mois=1
    fi

    # Calcul mois
    diff_mois=$(( date_actu[1] - date_args[1] - retenue_mois ))

    if [[ $diff_mois -lt 0 ]]; then
        diff_mois=$(( diff_mois + 12 ))
        retenue_annee=1
    fi

    # Calcul années
    diff_annee=$(( date_actu[2] - date_args[2] - retenue_annee ))

    # Cas 0 : Date argument est postérieure ou égale à la date courante
    if [[ $diff_annee -lt 0 ]] || [[ $diff_annee -eq 0 && $diff_mois -lt 0 ]]
|| [[ $diff_annee -eq 0 && $diff_mois -eq 0 && $diff_jour -le 0 ]]; then
        echo 0
    # Cas 1 : Antérieure de plus d'un an
    elif [[ $diff_annee -ge 1 ]]; then
        echo 1
    # Cas 2 : Antérieure de plus d'un mois et moins d'un an
    elif [[ $diff_mois -ge 1 ]]; then
        echo 2
    # Cas 3 : Antérieure de moins d'un mois
    else
        echo 3
    fi
fi
```